DETAILED TECHNICAL ARCHITECTURE

Northstar Wealth Copilot

Personal Finance & Tax-Loss Harvesting Demonstration

A safety-first, multi-agent application for statement ingestion, spending intelligence, portfolio drift analysis and guarded Alpaca paper trading.

Architecture baseline: Phase 2 local implementation
Document date: 1 September 2026
Branch: nidhi
Important: Educational demonstration; simulated paper trading only; not tax or investment advice.

1. Executive summary

Northstar ingests synthetic bank and brokerage statements, maintains a relational financial ledger, enriches assets with provider data, detects unusual spending and portfolio drift, generates tax-loss candidates, and subjects every candidate to deterministic safety gates. Only persisted final decisions are shown to users. A paper order can be submitted only through a separate, explicit, human-confirmed workflow.

> **Primary design principle:** the LLM may route and explain, but it cannot calculate authoritative financial facts, approve a candidate, bypass a hard gate, or submit an order.

IMPLEMENTED

Deterministic core

Parsing, anomaly features, portfolio mathematics, harvesting, wash-sale checks, risk controls, ranking and persistence.

IMPLEMENTED

Application surfaces

Streamlit UI, FastAPI REST endpoints, embedded FastMCP server, CLI jobs and provider-verification commands.

IMPLEMENTED

External integrations

Alpha Vantage, CoinGecko, Frankfurter, OpenAI and two Alpaca paper-account aliases.

PLANNED

AWS deployment

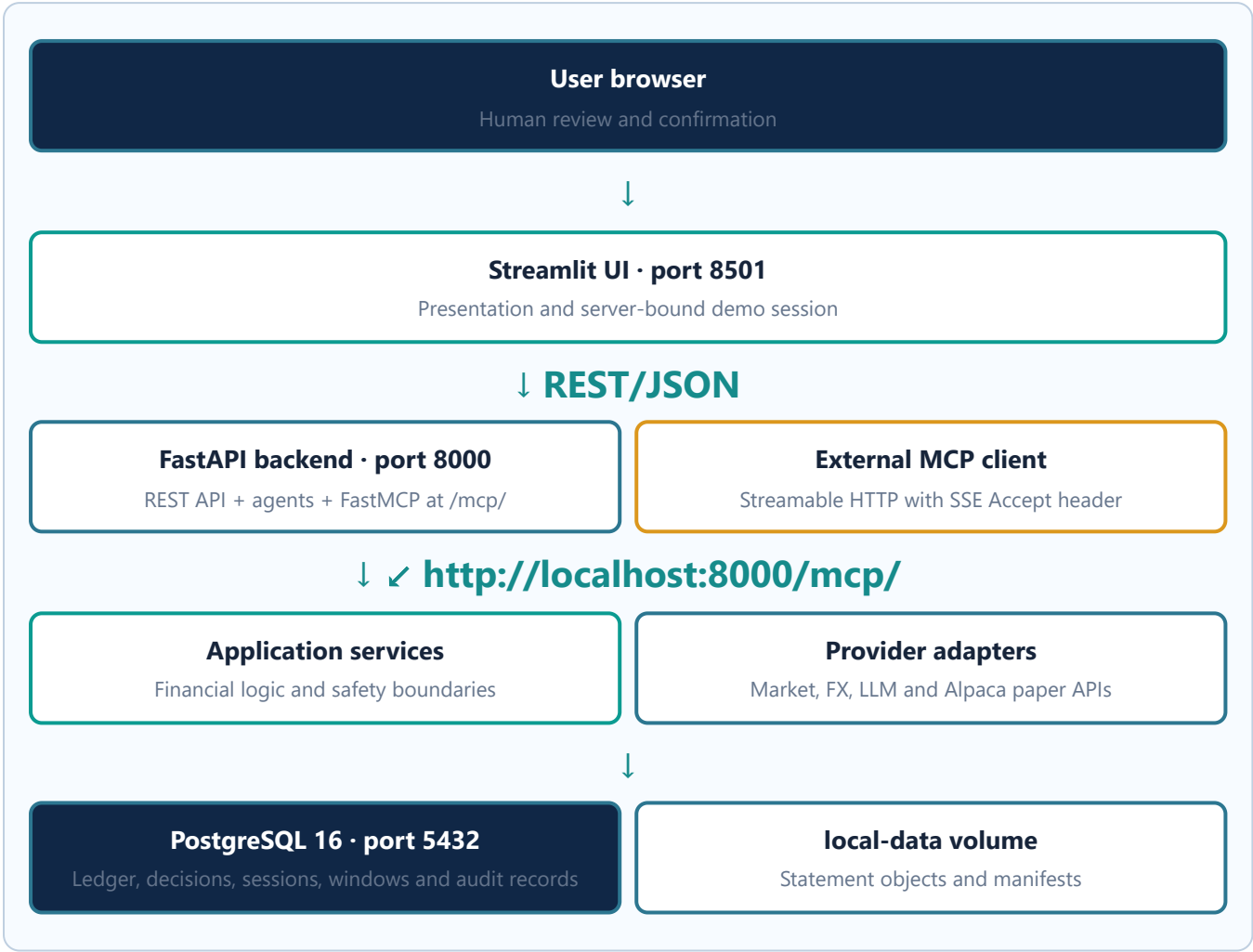
RDS/Postgres, DynamoDB rolling windows, S3 statement storage and protected container hosting. No scheduler is required for the submitted scope.

2. Problem-to-capability mapping

Problem requirement	Architecture response	Status
Ingest bank/brokerage PDFs	PyMuPDF deterministic parsers, validation, duplicate detection and persisted statements/transactions/tax lots.	Implemented locally
Use live market data	Provider router: equities/ETFs, crypto and FX adapters with normalized identifiers, timestamps and freshness checks.	Implemented; credential/rate dependent
Detect abnormal spending	Persisted feature pipeline and seeded IsolationForest with minimum-history controls.	Implemented
Measure portfolio drift	Current weights compared with persisted target allocations and risk profiles.	Implemented
Propose safe harvesting trades	Candidate generation followed by independent deterministic evaluation; rejected candidates remain auditable.	Implemented
Prevent wash-sale/risk violations	Fail-closed hard gates run during analysis and again at prepare/confirm.	Implemented and tested

Multi-agent separation	Orchestrator, Doc Parsing, ML Analysis and Eval definitions use controlled tools; services remain authoritative.	Implemented
MCP tools	FastMCP Streamable HTTP mounted inside the backend at `/mcp/`.	Implemented
AWS data architecture	Local Postgres/storage abstractions have AWS-oriented DynamoDB/S3 paths; infrastructure deployment remains.	Partially implemented / deployment pending

3. Current local deployment topology



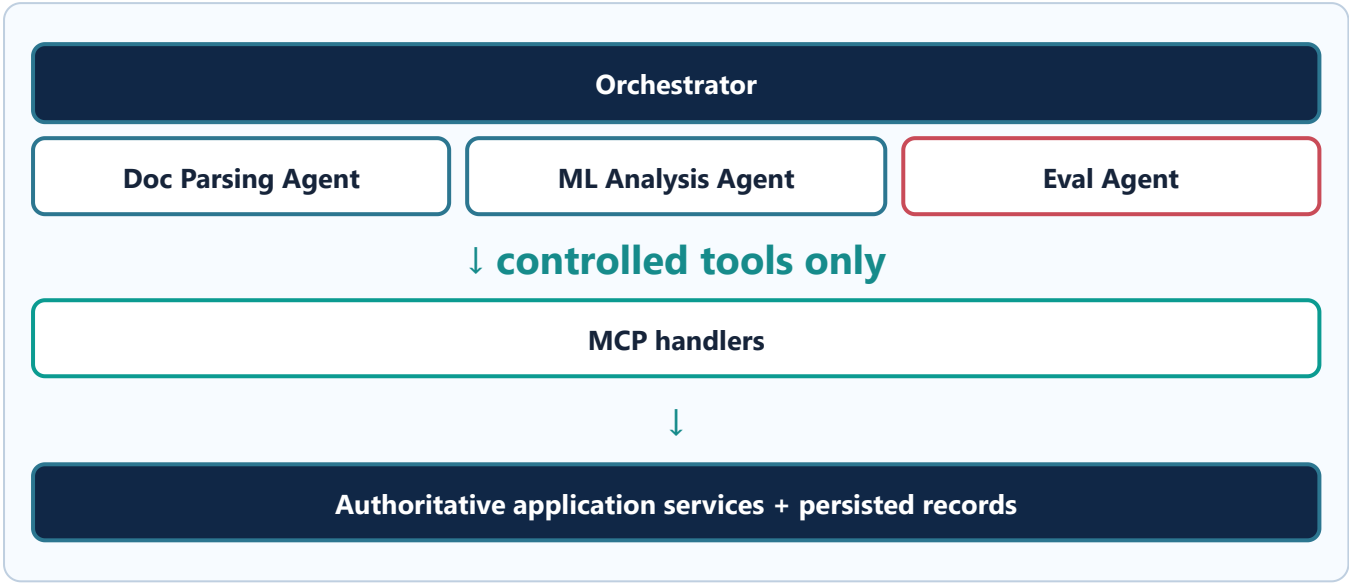
The default Compose topology contains three always-on services: **postgres**, **backend** and **ui**. MCP is deliberately restored to the backend container. An optional WhatsApp tunnel profile remains in Compose but its UI navigation entry has been removed.

4. Layered application architecture

Layer	Responsibility	Key modules
Domain	Enums, typed results and business errors; no web or LLM dependencies.	`app/domain`
Services	Authoritative ingestion, statistics, analysis, conflicts, harvesting, evaluation, portfolio calculations, sessions and paper execution.	`app/services`
Persistence	Async SQLAlchemy models, session factory and Alembic migrations.	`app/persistence`, `alembic`
Providers	Protocols and fake/live adapters. Providers supply data; they never decide safety.	`app/providers`
Adapters	Statement object storage and interchangeable rolling-window stores.	`app/adapters`
Parsers	Deterministic bank/brokerage PDF extraction and validation.	`app/parsers`

Agents	Routing and human-readable explanation using tool results.	`app/agents`
MCP	Typed, read/analysis-oriented wrappers around application services.	`app/mcp`
API	Health, upload, analysis, sessions, candidate preparation/confirmation and order refresh.	`app/api`
UI	Professional Streamlit presentation; no duplicated financial rules.	`app/ui`
Jobs/demo	Thin CLI entry points and deterministic fixture generation.	`app/jobs`, `app/demo_data`

5. Multi-agent and MCP design



Agent responsibilities

- **Orchestrator:** routes statement, spending, risk, drift and harvesting questions; maintains bounded conversation state; retrieves fresh authoritative results.
- **Doc Parsing Agent:** invokes the deterministic parsing pipeline. It does not invent missing values.
- **ML Analysis Agent:** initiates analysis and explains anomaly/drift outputs; it does not approve candidates.
- **Eval Agent:** exposes persisted deterministic evaluations and cannot replace a rule result with LLM opinion.

MCP surface

Representative tools include `get_quote`, `get_holdings`, `get_transactions`, `parse_statement`, `get_portfolio_insights`, `run_analysis`, spending statistics and `get_paper_order_status`. Submission and confirmation tools are explicitly forbidden.

> **Critical boundary:** MCP and agents cannot call `submit_paper_order` or `confirm_paper_order`. Only the guarded REST/UI workflow can reach paper execution.

Conversation state

Conversation sessions and items are persisted separately from financial truth. Memory can preserve conversational continuity, but balances, transactions, holdings, quotes and decisions are retrieved again through services/tools for authoritative answers.

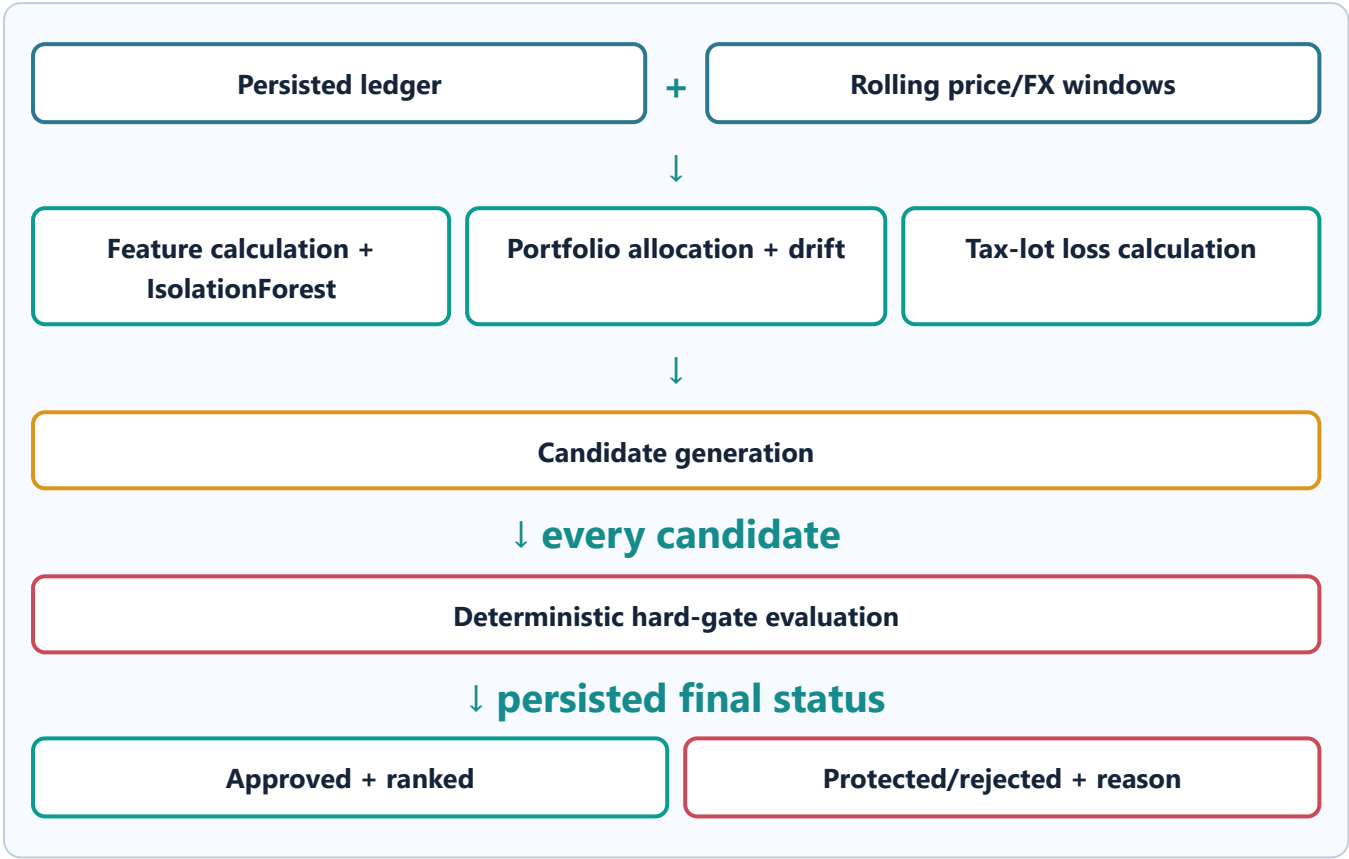
6. Statement ingestion flow

1. User uploads a PDF through Streamlit.
2. FastAPI validates file type/size and binds the request to the demo user.
3. Document type is recognized and a deterministic bank or brokerage parser extracts rows.

4. Schema, numeric, currency, date and reconciliation checks reject malformed or inconsistent data.
5. File identity prevents duplicate ingestion; duplicates return the existing statement record.
6. Normalized transactions, holdings, purchases, sales, dividends and tax lots are committed atomically.
7. Raw/local objects remain separate from normalized relational records.

> **Fail-closed parsing:** missing basis is not inferred, malformed financial values are rejected, and conversation memory never substitutes for document evidence.

7. Analysis and evaluation pipeline



Anomaly detection

Feature calculation uses deterministic transaction history, seeded IsolationForest configuration, minimum-history thresholds and persisted scores. Flags indicate transactions requiring review; they are not proof of fraud.

Portfolio drift

Holdings are valued, grouped by asset class and compared with target allocations. Risk limits and drift tolerances are stored as data rather than embedded in the UI.

Candidate generation and ranking

Losses are computed from reference quote, quantity and known basis. Candidates below the configured minimum usable loss remain visible as protected decisions. Approved candidates are ranked only after all hard gates pass.

Representative rejection codes

Control	Meaning for a stakeholder
WASH_SALE_CONFLICT	A known equity purchase conflicts with the 30-day wash-sale window.
SUBSTANTIALLY_IDENTICAL_REPLACEMENT	The proposed replacement is treated as too similar.
CRYPTO_REPURCHASE_CONFLICT	Blocked by the project's conservative crypto repurchase policy, not stated as tax law.

RISK_PROFILE_VIOLATION	The resulting allocation would violate the user's configured tolerance.
MISSING_BASIS	Cost basis is absent and is not guessed.
STALE_QUOTE	The reference price exceeds the freshness limit.
INSUFFICIENT_MIRROR_QUANTITY	The mapped Alpaca paper account cannot safely execute the proposed quantity.
PROFITABLE_LOT / BELOW_THRESHOLD	The lot is not a meaningful harvest candidate.

8. Paper-trading execution architecture

1. Analysis produces a persisted **APPROVED** candidate.
2. User chooses an equity/ETF candidate in the web application.
3. `prepare` reruns critical gates and creates a short-lived, read-only snapshot plus single-use token.
4. UI displays account, paper alias, symbol, SELL quantity, reference price, basis, proceeds and estimated loss in plain English.
5. User must select an initially unchecked review control; the confirmation button remains disabled until guards pass.
6. `confirm` verifies session binding, token, expiry, snapshot integrity, approval, freshness, quantity and paper-only configuration.
7. Only then may `PaperExecutionService` submit an idempotent SELL to the mapped Alpaca paper account.
8. User explicitly refreshes status; fill price and timestamps are stored separately from the evaluation reference quote.

> **Operational switches:** `ALPACA_PAPER=true` is mandatory. `ENABLE_PAPER_ORDERS=false` prevents submission while still allowing review. Replacement BUYs are display-only.

Paper mirror setup purchases are separate operational activity recorded as `PAPER_MIRROR_SETUP`. They establish executable inventory but never modify synthetic tax lots or realized tax history.

9. Provider architecture and data ownership

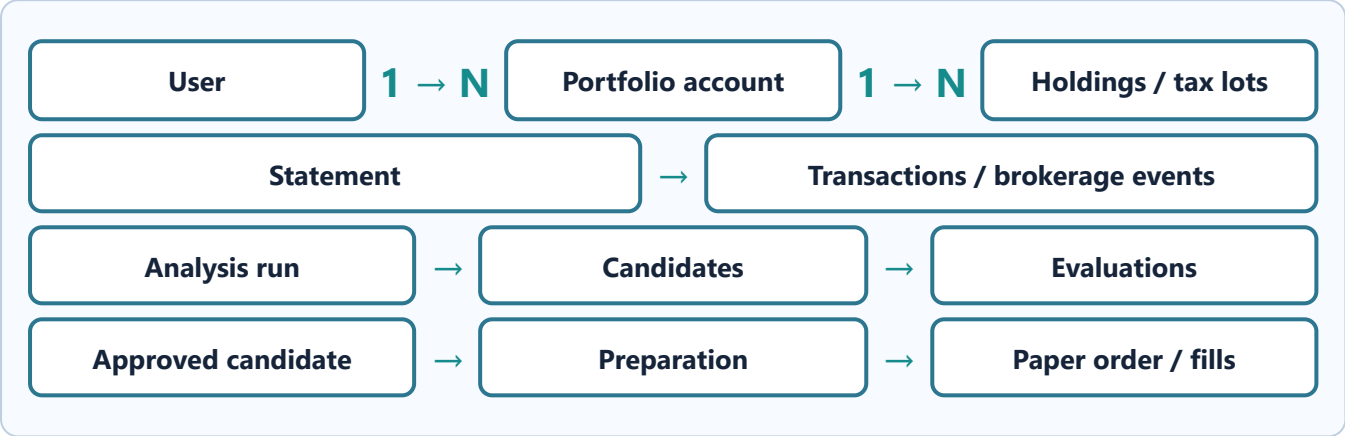
Data / capability	Owner	Controls
Equity/ETF reference prices	Alpha Vantage; Alpaca market-data fallback is implemented for live freshness scenarios	Timestamp, provider attribution, rate-limit handling
Crypto prices	CoinGecko	Explicit canonical-to-CoinGecko ID mapping
FX conversion	Frankfurter	Effective-date handling for weekends/holidays
Paper holdings/orders/fills	Alpaca paper API	Forced paper mode, alias mapping, asset-class and quantity checks
Financial ledger and evaluations	PostgreSQL application database	Relational integrity and audit history
LLM explanation/routing	OpenAI Agents SDK	Tool allowlist, deterministic fallback and no execution authority

Provider errors fail closed. Automated tests mock all provider responses; credential-dependent checks are executed separately with `app.verify_integrations`.

10. Data architecture

Relational ledger

PostgreSQL stores users, portfolio accounts, assets, statements, transactions, holdings, tax lots, brokerage events, risk profiles, target allocations, analysis runs, candidates, evaluations, conflicts, preparations, paper orders, mirror manifests/activity, anomaly scores, rolling-window records and agent/demo sessions.



Rolling-window contract

- `QUOTE#ASSET#CURRENCY`
- `PRICE_WINDOW#ASSET#CURRENCY`
- `ANOMALY_WINDOW#USER#FEATURE`
- `FX#BASE#QUOTE#DATE`
- `WINDOW_META#WINDOW_KEY`

Local mode stores windows in PostgreSQL. AWS mode provides a DynamoDB adapter with equivalent logical keys. Synchronization overlaps prior data by a configured interval, and metadata advances only after successful refresh.

Statement storage

Local mode uses the mounted `local-data` directory. The AWS target replaces this with S3 while retaining Postgres references and content identity. Raw statements are never embedded in agent conversation memory.

11. API and user-interface surfaces

Surface	Representative operations	Authority
REST API	Health, demo sessions, statement upload, holdings, analyses, candidate decisions, prepare/confirm, refresh, orchestrator chat	Calls application services
MCP	Quotes, holdings, transactions, parsing, statistics, portfolio insights, analysis and final decision retrieval	No confirmation/submission tools
Streamlit	Portfolio overview, uploads, questions, anomalies, analysis, drift, candidates, evaluations and paper orders	Presentation and explicit human interaction

CLI	Seed, run analysis, sync Alpaca, mirror preview/setup, provider verification	Operational wrappers, not duplicate business logic

Nontechnical UI cards translate identifiers and raw decimals into investment, account, proposed action, estimated proceeds/loss and safety explanations. The full JSON remains available only as a technical audit record.

12. Security, privacy and safety model

Secrets

Environment files are local and must never be committed. API keys, access tokens and signing secrets are excluded from UI and audit exports.

Session binding

A random server-side demo-session binding prevents cross-session order confirmation. It is explicitly not production authentication.

Least authority

Only PaperExecutionService may submit a paper SELL. Agents and MCP are excluded from confirmation/submission.

Idempotency

Analysis keys, preparation tokens and Alpaca client order IDs prevent accidental duplicate processing.

Source separation

Reference quotes, Alpaca fills, synthetic tax history and mirror setup activity remain distinct.

Fail closed

Missing, stale, inconsistent or unauthorized data produces a protected/rejected result—not a best guess.

Known production gaps

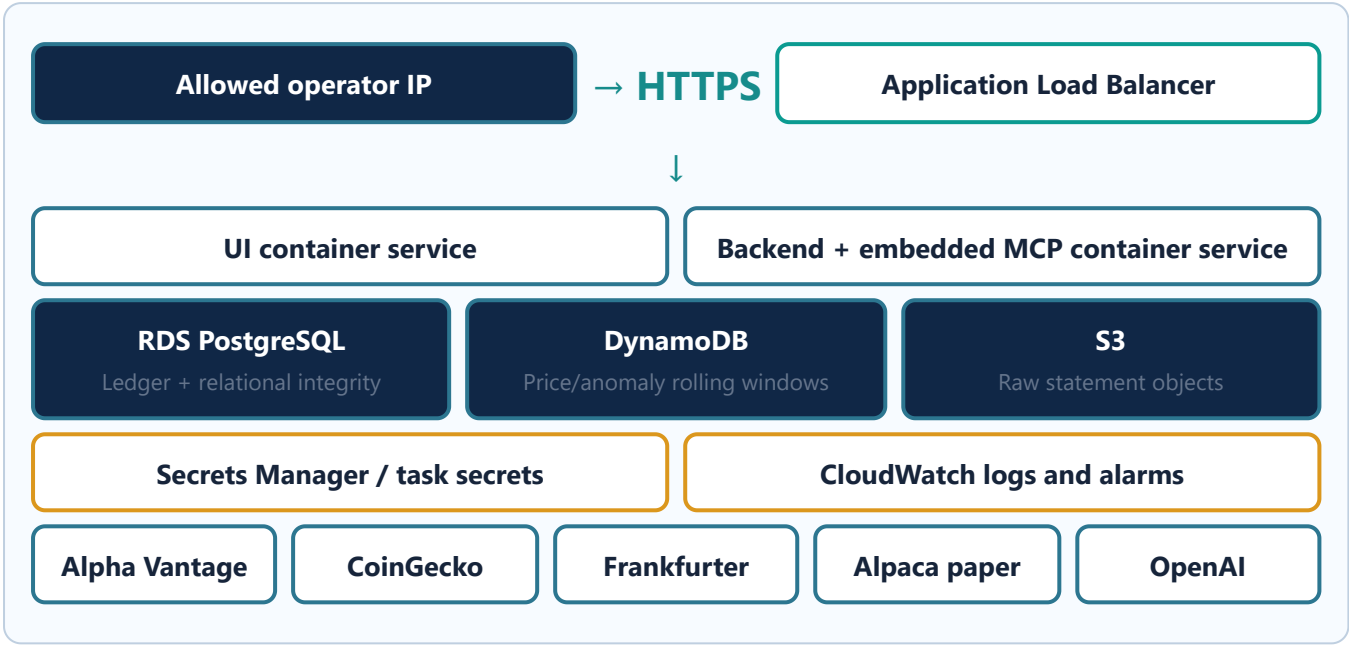
- The local demo session is not user authentication.
- AWS network controls, managed secrets, TLS, IAM, backups and monitoring are not yet deployed.
- Tax-loss calculations are educational and not a complete tax-return engine.
- The 30-day crypto policy is conservative project policy, not presented as current tax law.
- LangSmith is not currently integrated; Docker logs, persisted audit tables, tests and GitHub issues provide present diagnostics.

13. Testing and quality strategy

Test group	Coverage
Unit	Features, thresholds, freshness, conflict fingerprints, drift, ranking, MCP/agent boundaries, confirmation state and CLI contract.
Parser	Bank/brokerage extraction, PDF validation, deterministic current-demo generation and reconciliation.
Service	Analysis, sessions, paper execution, Alpaca sync, reconciliation, statistics and mirror seeding/idempotency.
Provider	Mocked HTTP/SDK adapters, mappings, parsing and error behavior.
Integration	Seed/ingest, persisted rolling windows and Phase 2 flows.
UI	Manual confirmation guards and nontechnical paper-order summary.

Current verified baseline: 79 automated tests passed after restoring embedded MCP. Live verification has succeeded for CoinGecko, Frankfurter and both Alpaca paper aliases; Alpha Vantage may return free-tier rate limits.

14. Target AWS architecture



The submitted scope intentionally excludes EventBridge, SQS, background workers and schedulers. Analysis remains user/CLI triggered unless stakeholders explicitly expand the scope.

Recommended deployment sequence

1. Provision private networking, RDS, S3 and DynamoDB with least-privilege IAM.
2. Store credentials outside images and source control.
3. Build and push immutable backend/UI container images.
4. Run Alembic migration as a controlled deployment task.
5. Deploy services behind TLS and IP restriction; expose MCP only where required.
6. Seed deterministic demo data without production credentials.
7. Run health/readiness, mocked smoke tests and credential-dependent provider checks.
8. Keep paper submission disabled by default; enable only for a supervised demonstration.

15. Operational runbook

Purpose	Command / endpoint
Start local system	`docker compose up --build -d`
Check containers	`docker compose ps`
Readiness	`http://localhost:8000/health/ready`
API documentation	`http://localhost:8000/docs`
MCP endpoint	`http://localhost:8000/mcp/` (MCP client; browser SSE error is expected)
Web application	`http://localhost:8501`

Full tests	`docker compose run --rm backend pytest`
Provider checks	`python -m app.verify_integrations --all`
Backend logs	`docker compose logs -f --tail=100 backend`

16. Completion assessment and next steps

Area	Assessment	Remaining evidence/work
Core financial application	Complete for demonstration scope	Maintain deterministic fixtures and tests.
UI and stakeholder presentation	Complete and professionally formatted	Run analysis after a UI/container restart to repopulate current session state.
MCP and agents	Implemented	MCP remains embedded in backend per restored design.
Live providers	Implemented	Alpha Vantage free-tier rate limits can block live checks.
Alpaca connectivity	Verified	Complete one supervised, manually confirmed paper SELL and refresh/fill evidence.
AWS	Not deployed	Provision, deploy, secure and perform cloud smoke tests.

> **Final position:** the project strongly aligns with the original problem statement locally. The major remaining deliverables are the supervised Alpaca end-to-end execution evidence and AWS deployment. Scheduling was intentionally omitted by stakeholder direction.

Appendix A — Key configuration controls

Variable	Purpose	Safe default
USE_LIVE_PROVIDERS	Select fake/deterministic or live market adapters.	false
ALPACA_PAPER	Enforces paper-trading endpoints.	true
ENABLE_PAPER_ORDERS	Global order-submission kill switch.	false
QUOTE_MAX_AGE_MINUTES	Rejects stale execution references.	15
MIN_LOSS_THRESHOLD	Minimum usable loss for approval.	50.00
DEMO_MODE / DEMO_AS_OF_DATE	Controls deterministic historical/current demo clock.	Fixed in tests
ISOLATION_FOREST_SEED	Makes anomaly-model results repeatable.	42

Prepared from the repository architecture, README, Compose topology, service/module structure and verified test baseline. No credentials or secret values are included.